Hardware Documentation

Editor: Kristian Kirov

rev 0.1 | 10.04.2026

1. Overview

1.1. Purpose of this document

This document describes the hardware setup and calibration work performed for the Remote Controlled Power Supply project on the Olimex A20 platform. It covers the initial A20 board preparation, UEXT/MOD-IO integration, ADC and relay testing, analog thermometer calibration, fan system validation, and hardware-to-backend interface development.

2. Hardware Setup

2.1. SD card preparation

2.1.1. Recommended Linux distro for Olimex A20

Olimex recommends flashing the image they provide on https://images.olimex.com/release/a20/ . For this project we use the current latest version (bookworm-base-20260323).

2.1.2. Flashing process

The flash process is pretty straight forward. For best experience use a micro SD card with speed rating between class 10 and A1. This project uses Samsung EVO Plus 64G (class A1).

In order to Flash the SD card we need a PC/Laptop that has software similar to WinRAR and BalenaEtcher. You need to extract the files from the file and then use BalenaEtcher to flash it.

When Flashing is done safely disconnect the SD card from the PC.

2.1.3. Boot validation

The Olimex board has an SD card holder. Insert the flashed card into the board. Hearing a click is a good sign.

For easier debug it is recommended to have a Monitor with a HDMI cable, keyboard, mouse and Ethernet access. Connect all I/O to the respected ports.

After IO is connected you can safely connect the 12V power supply to the board. At this point you should see a few LED lights.

Initial boot will take between 2-4 minutes.

2.1.4. A20 board initial setup

Package update: sudo apt update
SSH access when connected to local network:

Command: ssh olimex@a20-olinuxino

Password: olimex

2.2. UEXT / MOD-IO connection

The MOD-IO board is connected to the Olimex A20 via the UEXT interface, which provides a standardized 10-pin connector for I2C, SPI, UART, and GPIO communication. Ensure the UEXT cable is securely plugged into the A20's UEXT port and the MOD-IO's UEXT socket, aligning the red wire (pin 1) on both sides for correct orientation.

2.2.1. Physical wiring diagram
2.2.2. Pin mapping and connector orientation
2.3. Verification procedure using relays
2.4. ADC and relay functional setup
2.5. Required libraries (smbus, etc.)
2.6. Test procedures for ADC reads
2.7. Relay on/off validation
2.8. Fan system setup
2.9. Relay-controlled fan wiring
2.10. Power supply requirements
2.11. Testing steps and observed limitations
2.12. Voltage divider circuit for ADC input
2.13. Circuit schematic
2.14. Component values
2.15. Verification and test results
3. Sensor Calibration
3.1. Analog thermometer setup
3.1.1. Initial behavior observed
3.2. Converting ADC values to Celsius
3.3. Calibration script usage
3.4. Example calibration data and formula

4. Software / Firmware / Scripts

4.1. Low-level hardware scripts

4.2. ADC reading scripts

4.2.1. single-read

4.2.2. continuous monitoring

4.3. Digital input/output scripts

4.4. Relay control scripts

4.5. How the hardware scripts expose data to the backend

4.6. Required dependencies and installation commands

5. Safety and Reliability

5.1. Hardware-level safety checks to implement

5.2. Fan activation on high temperature

5.3. Emergency cutoff considerations

5.4. Current status of safety features

5.5. Recommended next steps

6. Hardware Status Summary

6.1. Completed tasks

6.2. Work in progress

6.3. Pending tasks

7. Appendices

7.1. Useful commands

7.2. Test logs and notes

7.3. References and links to relevant docs

7.3.1. Project Github:

https://github.com/Denislav30/RemoteControlledPowerSupply

7.3.2. Olimex Links:

https://www.olimex.com/Products/Modules/IO/MOD-IO/open-source-hardware

https://github.com/OLIMEX/OLINUXINO/blob/master/DOCUMENTS/OLIMAGE/Olimage-guide.pdf

https://www.olimex.com/Products/OLinuXino/A20/A20-OLinuXino-LIME2/open-source-hardware

1. How to connect:
   ssh olimex@a20-olinuxino
   Password: olimex

2. Check if Mod IO is connected
   a. sudo i2cdetect -y 0/1/2
   b. Search for 58
3. Test Mod IO by flipping the ALL RELAYS
   a. sudo i2cset -y X 0x58 0x10 0x0F

1. Toggle the Relays

The MOD-IO has 4 relays. You can control them by sending a bitmask to command 0x10.

Turn ON Relay 1:
sudo i2cset -y 2 0x58 0x10 0x01

Turn ON Relay 2:
sudo i2cset -y 2 0x58 0x10 0x02

Turn ON ALL Relays:
sudo i2cset -y 2 0x58 0x10 0x0F

Turn OFF ALL Relays:
sudo i2cset -y 2 0x58 0x10 0x00

Note: If you are using I2C bus 1 instead of 2, change the 2 in the command to a 1. You should hear a distinct "click" from the board when you run these.

2. Read Digital Inputs

If you have switches or sensors connected to the opto-isolated inputs, you can check their status using command 0x20.

Tell the board you want to read the inputs:
sudo i2cset -y 2 0x58 0x20

Read the result:
sudo i2cget -y 2 0x58

The output will be a hex value. For example, 0x01 means Input 1 is active, 0x03 means Inputs 1 and 2 are active, etc.

3. Read Analog Inputs

The MOD-IO has 4 analog inputs (0-3.3V). Each has its own command:

AIN1: 0x30 | AIN2: 0x31 | AIN3: 0x32 | AIN4: 0x33

To read AIN1:
sudo i2cset -y 2 0x58 0x30
sudo i2cget -y 2 0x58